

Internet Applications — Theory ↔ Code Correlation

MovieLens Explorer (FastAPI + SQLite backend · vanilla HTML/CSS/JS frontend)

Course: *Διαδίκτυο και Εφαρμογές* — mapping every lecture concept to the assignment code

Spring 2026

Contents

1	How to read this document	3
1.1	Project map	3
1.2	Request → response, end to end	3
2	Lecture 1 — Introduction (WWW, client/server, REST, architecture)	4
2.1	Client-Server model · Front-End vs Back-End	4
2.2	Static vs Dynamic pages · Rich Internet Application (RIA)	4
2.3	HTTP vs HTML	4
2.4	REST and its guiding principles	4
2.5	Architecture tiers	5
2.6	Data-interchange formats: JSON (vs XML, YAML)	5
3	Lecture 2 — HTTP	6
3.1	HTTP is stateless	6
3.2	HTTP methods (verbs)	6
3.3	URL anatomy: path, query, percent-encoding	6
3.4	MIME types · request body	6
3.5	Status codes	7
3.6	Same-Origin Policy and CORS	7
3.7	Testing with curl	7
4	Lecture 3 — JavaScript	8
4.1	The runtime: single-threaded, event-driven, non-blocking	8
4.2	const vs let · block scope	8
4.3	Objects as associative arrays · bracket notation	8
4.4	Strict vs loose equality · truthy / falsy	8
4.5	Type conversion (typecasting)	8
4.6	Functions are first-class · they always return something	9
4.7	Closures and lexical scope	9
4.8	async / await and Promises (<i>the headline networking topic</i>)	9
4.9	Events and event handlers · the DOM	10
5	Lecture 4 — Server-side Programming & REST Services	11
5.1	FastAPI: app and endpoints	11
5.2	Endpoint input: path, query, and body	11
5.3	Request-body validation with Pydantic models	11
5.4	HTTP as an API: verbs → CRUD	12
5.5	Uvicorn and ASGI	12
5.6	SQLite: connection, cursor, the with statement	12
5.7	DDL: CREATE TABLE · storage classes · primary keys	12

5.8	Parameterized queries → SQL-injection safety	13
5.9	Bulk insert with executemany · transactions · commit	13
5.10	Querying: fetchall · row → dict · JOIN · GROUP BY	13
5.11	REST client from JavaScript: fetch + Promises	14
5.12	“Stateless compute”: the recommendation endpoint	14
6	Cross-cutting: the recommendation algorithm (domain logic)	15
7	Concept → code quick-reference table	16
8	Appendix: syllabus concepts deliberately <i>not</i> used	17

1 How to read this document

This document connects **the four course lectures**

1. *Introductory Lecture* (WWW, client/server, REST overview, architecture),
2. *HTTP*,
3. *JavaScript*, and
4. *Server-side Programming & REST Services*

to **the exact place in the MovieLens code where each idea is used**.

For every concept you get three things:

- **Theory** — a short restatement of what the lecture says.
- **Where in the code** — the file + function, with a focused snippet.
- **Exam note** (*when useful*) — the “why” most likely to be asked.

The same explanations are also embedded **inline in the source files** as THEORY · L<lecture> · <concept> comments, so the code reads as a study guide.

1.1 Project map

backend/	
src/	
main.py	FastAPI app: CORS, lifespan, router mounting, uvicorn
models.py	Pydantic request models (body validation)
db.py	sqlite3 connection (with-statement context manager)
setup_db.py	CSV → SQLite ETL: CREATE TABLE, executemany, commit
reset_db.py	drop + rebuild the DB
recommender.py	user-based collaborative filtering (domain logic)
routes/	
movies.py	GET /movies (query), GET /ratings/{id} (path), POST /movies
recommendations.py	POST /recommendations (stateless compute)
tags.py	POST /tags/movies (JOIN + GROUP BY)
frontend/	
index.html	structure (markup) + inline event handlers + <script>
index.css	presentation (style-sheet language)
index.js	behaviour: fetch/async-await, DOM, events, closures

1.2 Request → response, end to end

Browser (index.html/.js/.css)	FastAPI server (uvicorn, port 3000)
onclick → searchMovies()	
fetch("/movielens/api/movies?search=toy")	— HTTP GET → @router.get("/movies")
(await: non-blocking)	SELECT ... LIKE ?
response.json()	← 200 application/json — {movies:[...]}
build HTML, set tbody.innerHTML	

2 Lecture 1 — Introduction (WWW, client/server, REST, architecture)

2.1 Client-Server model · Front-End vs Back-End

Theory. A web application is split into *client-side code* (HTML, CSS, JS running in the browser) and *server-side code* (programs + DB running on another machine), talking over HTTP. The browser submits requests; the server services them.

Where in the code. The whole repository is this split: `frontend/` is the client (downloaded and executed in the browser), `backend/` is the server (Python/FastAPI + SQLite). They communicate only over HTTP via one constant:

```
// frontend/index.js
const API_BASE = "http://localhost:3000/movielens/api";
```

2.2 Static vs Dynamic pages · Rich Internet Application (RIA)

Theory. A *static* page never changes between sessions; a *dynamic* page's content changes. Web 2.0 created RIAs — pages that behave like desktop apps, where parts of the page interact with services and change their content/look without a full reload.

Where in the code. `index.html` is served as-is (static shell), but its tables are filled at runtime by JS rewriting the DOM — no page reload:

```
// frontend/index.js — searchMovies()
tbody.innerHTML = html; // re-render just the results, page stays put
```

2.3 HTTP vs HTML

Theory. HTTP is an *application-layer protocol* that defines the structure and meaning of exchanged data. HTML is a *markup language* that defines how data is displayed. They are different things that cooperate.

Where in the code. HTML lives in `index.html` (structure); HTTP is the transport used by every `fetch()` call in `index.js` and answered by FastAPI.

2.4 REST and its guiding principles

Theory. REST (Representational State Transfer) is an *architectural style* with constraints: **Client-Server**, **Stateless** (each request carries all it needs), **Cacheable**, **Uniform Interface** (URIs + HTTP verbs + status codes), **Layered System**, and optional **Code-on-Demand**. It typically uses JSON over HTTP.

Where in the code. The backend is a REST API:

- *Uniform Interface*: resources under one URI prefix, addressed with HTTP verbs.

```
# backend/src/main.py
API_PREFIX = "/movielens/api"
app.include_router(movies.router, prefix=API_PREFIX)
```

- *Stateless*: `/recommendations` stores nothing — it reads the body, computes, replies (see Lecture 4 → *Stateless compute*).
- *Client-Server*: GUI (frontend) is fully separated from data storage (SQLite).

2.5 Architecture tiers

Theory. Systems are layered: *presentation, application/business logic, resource manager (DB)*. A **3-tier** architecture keeps all three distinct.

Where in the code. This project is a clean 3-tier app:

Tier	Responsibility	In this project
Presentation	UI	frontend/ (HTML/CSS/JS)
Application	business logic	FastAPI routes + <code>recommender.py</code>
Resource manager	persistence	SQLite via <code>db.py</code> / <code>setup_db.py</code>

2.6 Data-interchange formats: JSON (vs XML, YAML)

Theory. JSON is a lightweight, language-independent text format built on two structures — *objects* (name/value pairs) and *arrays*. It is the usual REST data format; its syntax is a subset of JavaScript object syntax.

Where in the code. Every response is JSON; the client serialises/parses JSON:

```
body: JSON.stringify({ title: title, genres: genres }) // JS object → JSON text
const data = await response.json();                 // JSON text → JS object
```

Exam note. XML, YAML, SOAP/WSDL/UDDI are covered in the syllabus but **not used here** — this app is REST/JSON. See *Appendix: syllabus concepts not used*.

3 Lecture 2 – HTTP

3.1 HTTP is stateless

Theory. Every request-response cycle is independent; no data is preserved between connections. State, if needed, must be re-supplied (cookies/sessions) or re-sent by the client.

Where in the code. The recommender takes the **entire** ratings list on every call because the server remembers nothing between requests:

```
// frontend/index.js – getRecommendations()
const data = await callApi("/recommendations", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ ratings: ratings }), // full session state, every time
});
```

The session “state” (your ratings) lives only in a browser variable `let myRatings = {}` and is gone on refresh — a deliberately client-side answer to HTTP statelessness.

3.2 HTTP methods (verbs)

Theory. GET = retrieve a resource; POST = send data to a resource; PUT = create/replace; DELETE = remove; OPTIONS = list supported methods (used by CORS preflight); etc.

Where in the code. The API uses GET and POST deliberately:

```
# backend/src/routes/movies.py
@router.get("/movies")           # GET → read (safe, no side effects)
@router.get("/ratings/{movie_id}") # GET → read
@router.post("/movies")          # POST → send data / create
```

3.3 URL anatomy: path, query, percent-encoding

Theory. A URL is `scheme://host:port/path?query#fragment`. The *query string* carries name=value pairs. Characters unsafe in a URL are *percent-encoded* (space → %20).

Where in the code. Search builds a query string and percent-encodes the user keyword:

```
// frontend/index.js – searchMovies()
const data = await callApi("/movies?search=" + encodeURIComponent(keyword));
//                               path      query      └ percent-encoding
```

The *path parameter* form is used for ratings:

```
const data = await callApi("/ratings/" + movieId); // id sits IN the path
```

3.4 MIME types · request body

Theory. A MIME (media) type `type/subtype` tells the receiver how to process the bytes (application/json, text/html). Only POST/PUT/PATCH carry a body; the Content-Type header names the body’s MIME type.

Where in the code. Every POST declares its body as JSON:

```
// frontend/index.js – addMovie()
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ title: title, genres: genres }),
```

3.5 Status codes

Theory. 2xx success, 3xx redirect, 4xx client error, 5xx server error (e.g. 201 Created, 404 Not Found, 500). REST maps outcomes onto these.

Where in the code. FastAPI returns 200 OK for our normal return {...} responses automatically, and emits 422 Unprocessable Entity on its own when a Pydantic model fails validation (e.g. a missing title in POST /movies) — without any code from us. That automatic 4xx is the theory in action.

3.6 Same-Origin Policy and CORS

Theory. An *origin* is the (scheme, host, port) tuple. The Same-Origin Policy blocks a script from one origin from reading responses from another — a security mechanism. **CORS** is the opt-in protocol (special response headers like Access-Control-Allow-Origin) that permits chosen cross-origin requests; non-simple requests are first *preflighted* with an OPTIONS request.

Where in the code. The frontend (e.g. file:// or localhost) and backend (localhost:3000) are **different origins**, so CORS must be enabled or the browser blocks every fetch:

```
# backend/src/main.py
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # Access-Control-Allow-Origin: *
    allow_methods=["*"],      # also answers the preflight OPTIONS
    allow_headers=["*"],
)
```

Exam note. Our POSTs send Content-Type: application/json, which is **not** a CORS-“simple” content type, so the browser issues a **preflight OPTIONS** first. The CORS middleware is what answers it — remove the middleware and the POSTs fail in the browser even though curl still works.

3.7 Testing with curl

Theory. HTTP can be exercised from the command line (curl) or tools like Postman, independent of any browser.

Where in the code. The API is curl-testable, e.g.:

```
curl "http://localhost:3000/movielens/api/movies?search=toy"
curl -X POST http://localhost:3000/movielens/api/movies \
  -H "Content-Type: application/json" \
  -d '{"title":"My Film (2026)","genres":"Comedy"}'
```

4 Lecture 3 — JavaScript

4.1 The runtime: single-threaded, event-driven, non-blocking

Theory. JS runs in an engine (V8) on a single thread with an *event loop*: the *stack* runs synchronous code, slow operations are delegated to *Web APIs*, and their callbacks wait in a *queue* until the stack is empty. This gives asynchronous, non-blocking I/O without multiple threads.

Where in the code. Every network call uses this model. `fetch` is handed to the browser's Web API; the function yields the thread and resumes when the response is ready — the UI never freezes:

```
// frontend/index.js
async function callApi(path, options) {
  const response = await fetch(API_BASE + path, options); // delegated, non-blocking
  return response.json();
}
```

4.2 const vs let · block scope

Theory. ES6 `let/const` are *block-scoped*. `const` is an immutable *binding* (cannot be reassigned); `let` is mutable. Prefer `const`.

Where in the code.

```
const API_BASE = "http://localhost:3000/movielens/api"; // never reassigned
let myRatings = {}; // reassigned/mutated as the user rates movies
```

4.3 Objects as associative arrays · bracket notation

Theory. A JS object is a collection of properties (name → value), usable as an associative array via bracket notation `obj[key]`.

Where in the code. `myRatings` and `searchedMovies` are maps keyed by movie id:

```
searchedMovies[movie.movieId] = movie; // set
myRatings[movieId] = { title: movie.title, rating: parseFloat(value) };
delete myRatings[movieId]; // remove a property
const ids = Object.keys(myRatings); // all keys
```

4.4 Strict vs loose equality · truthy / falsy

Theory. `===` compares value **and** type (no coercion); `==` coerces. *Falsy* values are `false`, `0`, `""`, `null`, `undefined`, `NaN`; everything else is *truthy*.

Where in the code. Form validation uses strict equality against the empty string (itself a falsy value):

```
if (title === "" || genres === "") { /* reject */ } // === : no coercion
if (value === "") { /* nothing chosen in the dropdown */ }
```

4.5 Type conversion (typecasting)

Theory. Values can be converted explicitly. DOM inputs and object keys are strings, so numbers must be parsed.

Where in the code.

```
rating: parseFloat(value) // "4.5" (string) → 4.5 (number)
movieId: parseInt(id) // object key string → integer for the API
```

4.6 Functions are first-class · they always return something

Theory. Functions are first-class citizens: assignable to variables, passable as arguments, returnable. A function always returns a value (undefined absent a return).

Where in the code. `ratingDropdown` builds and **returns** an HTML string that the caller embeds:

```
function ratingDropdown(movieId) {  
  let html = "<select id='rating-" + movieId + "'>";  
  for (let r = 0.5; r <= 5; r += 0.5) { html += "<option ...>"; }  
  return html;           // a value, consumed by searchMovies()  
}
```

4.7 Closures and lexical scope

Theory. A *closure* is a function bundled with the lexical environment in which it was created — it keeps access to outer-scope variables even after the outer function returned. Closures let you associate data with the functions that operate on it (much of async/event-driven JS relies on this).

Where in the code. `rateMovie`, `searchMovies`, `getRecommendations`, `removeRating` are all closures over the module-level state `myRatings` / `searchedMovies`. Each event handler keeps read/write access to that shared data across unrelated clicks:

```
let myRatings = {};           // captured by the closures below  
function rateMovie(movieId, value) {  
  const movie = searchedMovies[movieId];    // reads outer state  
  myRatings[movieId] = { title: movie.title, rating: parseFloat(value) };  
  showMyRatings();  
}
```

4.8 async / await and Promises (the headline networking topic)

Theory. A **Promise** represents the future result of a single asynchronous computation; `.then()` runs on success, `.catch()` on failure, `.finally()` always. `fetch()` is a **Promise-based** HTTP API. `async/await` is syntactic sugar over promise chaining: an `async` function returns a Promise, and `await` pauses it until the awaited Promise settles, yielding the resolved value.

Where in the code. `callApi` is the canonical example, and `try/catch` is the `async` form of `.catch()`:

```
async function callApi(path, options) {           // returns a Promise  
  const response = await fetch(API_BASE + path, options); // await the request  
  return response.json();                         // .json() is itself a Promise  
}  
  
// caller:  
try {  
  const data = await callApi("/movies", { method: "POST", /* ... */ });  
} catch (error) {                                // = .catch() for the rejection  
  feedback.textContent = "Error: could not reach the server.";  
}
```

The equivalent `.then()` chain from the lecture would be `fetch(url).then(r => r.json()).then(data => ...).catch(err => ...)`.

4.9 Events and event handlers · the DOM

Theory. JS registers callbacks for events (mouse/keyboard/system). Two ways: inline onevent HTML attributes, or `addEventListener`. The *DOM* (a Web API) is the live object tree of the page that scripts read and modify.

Where in the code. Inline handlers in the markup invoke JS functions; the functions then read inputs and rewrite the DOM:

```
<!-- frontend/index.html -->
<button onclick="searchMovies()">Search</button>
```

```
// frontend/index.js
const keyword = document.getElementById("search-input").value; // read DOM
tbody.innerHTML = html;                                     // write DOM
```

5 Lecture 4 — Server-side Programming & REST Services

5.1 FastAPI: app and endpoints

Theory. FastAPI is a modern high-performance Python web framework for APIs, built on **Pydantic** (validation) and **Starlette** (ASGI), designed around `async/await`, with auto-generated interactive docs (Swagger UI / ReDoc). You create an app, then declare endpoints with decorators.

Where in the code.

```
# backend/src/main.py
app = FastAPI(title="MovieLens Backend", lifespan=lifespan)
```

```
# backend/src/routes/movies.py
router = APIRouter()
@router.get("/movies")
def search_movies(search: Optional[str] = None): ...
```

Visit <http://localhost:3000/docs> to see the auto-generated Swagger UI the lecture mentions.

5.2 Endpoint input: path, query, and body

Theory. Input reaches an endpoint three ways: the URL **path** (`/users/1`), the URL **query string** (`/users?start=0&limit=10`), or the request **body** (JSON, via POST).

Where in the code. All three appear:

```
# QUERY parameter – not in the path ⇒ FastAPI treats it as ?search=...
@router.get("/movies")
def search_movies(search: Optional[str] = None): ...

# PATH parameter – {movie_id} in the route, typed as int
@router.get("/ratings/{movie_id}")
def get_ratings(movie_id: int): ...

# BODY – a Pydantic model parameter
@router.post("/movies")
def add_movie(movie: MovieAdd): ...
```

5.3 Request-body validation with Pydantic models

Theory. Declaring a body as a Pydantic model makes FastAPI read the JSON, convert types, validate, hand you a typed object, and publish a JSON Schema in the OpenAPI docs — all automatically.

Where in the code.

```
# backend/src/models.py
class MovieAdd(BaseModel):
    title: str
    genres: str

class RatingInput(BaseModel):
    movieId: int
    rating: float
```

```
class RecommendationRequest(BaseModel):
    ratings: List[RatingInput] # validates a JSON array of objects
```

Exam note. Typing the body `RecommendationRequest` (instead of a bare `list[dict]`) is what gives free 422 errors on malformed input and self-doc in `/docs`. This was the original bug fix.

5.4 HTTP as an API: verbs → CRUD

Theory. In REST, HTTP “acts as an API.” The verbs map to CRUD: POST→Create, GET→Read, PUT→Update, DELETE→Delete; success is 2xx, failure 4xx/5xx. A guiding principle is to **respect the original meaning of the verbs**.

Where in the code. `routes/movies.py` demonstrates the mapping directly:

```
@router.get("/movies") # Read
@router.get("/ratings/{movie_id}") # Read
@router.post("/movies") # Create → returns the new movieId
```

5.5 Uvicorn and ASGI

Theory. **ASGI** (Asynchronous Server Gateway Interface) is the standard contract between async Python apps and the network. **Uvicorn** is an ASGI server (uvloop + httptools) that handles HTTP requests asynchronously.

Where in the code.

```
# backend/src/main.py
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=3000) # host 0.0.0.0 → on the LAN
```

5.6 SQLite: connection, cursor, the with statement

Theory. SQLite is an embedded DB (a C library, no server) with a dynamic type system (storage classes NULL/INTEGER/REAL/TEXT/BLOB). In Python you connect, get a **cursor** to run SQL, and you should **close** the connection to free resources — best done with the with statement.

Where in the code.

```
# backend/src/db.py
@contextmanager
def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row # rows accessible by column name → dict()
    try:
        yield conn
    finally:
        conn.close() # always freed
```

Usage everywhere: `with get_db() as conn: cursor = conn.cursor()`.

5.7 DDL: CREATE TABLE · storage classes · primary keys

Theory. DDL defines the schema. SQLite storage classes are general; any column (except an INTEGER PRIMARY KEY) can hold any class. Booleans/dates have no dedicated class.

Where in the code.

```
# backend/src/setup_db.py
cursor.execute("""
    CREATE TABLE IF NOT EXISTS ratings (
        userId    INTEGER,
        movieId   INTEGER,
        rating     REAL,
        timestamp  INTEGER,
        PRIMARY KEY (userId, movieId)  -- one rating per (user, movie)
    )
""")
```

movies.movieId INTEGER PRIMARY KEY makes SQLite auto-assign ids, so POST /movies gets a unique id for free (cursor.lastrowid).

5.8 Parameterized queries → SQL-injection safety

Theory. Use placeholders (? positional or :name named) so user data is *bound*, never concatenated into SQL — the standard defence against SQL injection.

Where in the code. Every user value is bound:

```
# backend/src/routes/movies.py
cursor.execute("SELECT * FROM movies WHERE title LIKE ?", (f"%{search}%",))
cursor.execute("INSERT INTO movies (title, genres) VALUES (?, ?)",
               (movie.title, movie.genres))
```

```
# backend/src/recommender.py – a safe, dynamically-sized IN list
placeholders = ",".join("? " for _ in movie_ids)          # "? , ? , ?"
cursor.execute(f"SELECT ... WHERE movieId IN ({placeholders})", movie_ids)
```

Exam note. Only the *number* of ? is built from the list length; the *values* are still bound. The keyword in tags.py is bound the same way even though the WHERE-clause text is chosen in Python.

5.9 Bulk insert with executemany · transactions · commit

Theory. executemany(sql, list_of_tuples) inserts many rows in one batched call. An INSERT implicitly opens a transaction that must be **committed** to persist; using the connection as a with block commits on success / rolls back on error.

Where in the code.

```
# backend/src/setup_db.py
with sqlite3.connect(DB_PATH) as conn:          # one transaction for the whole load
    cursor = conn.cursor()
    cursor.executemany(
        "INSERT INTO movies (movieId, title, genres) VALUES (?, ?, ?)", movies)
    cursor.executemany(
        "INSERT INTO ratings (userId, movieId, rating, timestamp) VALUES (?, ?, ?, ?)",
        ratings)
    conn.commit()
```

5.10 Querying: fetchall · row → dict · JOIN · GROUP BY

Theory. SELECT returns rows; fetchall() returns all of them, fetchone() one at a time (or None). JOINS combine tables; GROUP BY aggregates.

Where in the code. The tag search joins two tables and aggregates:

```
# backend/src/routes/tags.py
cursor.execute(f"""
    SELECT m.movieId, m.title, m.genres, MIN(t.tag) AS matchingTag
    FROM tags AS t
    JOIN movies AS m ON m.movieId = t.movieId
    WHERE {where_clause}
    GROUP BY m.movieId, m.title, m.genres
    ORDER BY m.title
""", params)
movies = cursor.fetchall()
return {"status": "success", "movies": [dict(movie) for movie in movies]}
```

5.11 REST client from JavaScript: fetch + Promises

Theory. A REST service can be called from client-side JS with the fetch API, which is **Promise-based**; errors are handled with `.catch()` (or `try/catch` under `async/await`). This is the lecture's closing topic, "Asynchronous JavaScript — `fetch(...)`".

Where in the code. This is exactly `callApi` + every handler in `index.js` (see Lecture 3 → *async/await and Promises*). It closes the loop: the JS client of Lecture 3 calls the FastAPI/SQLite server of Lecture 4 over the HTTP of Lecture 2.

5.12 "Stateless compute": the recommendation endpoint

Theory. REST services should be stateless and (best practice) each operation a stateless function call taking inputs and returning outputs.

Where in the code. `POST /recommendations` performs **zero writes** — it is a pure function of its input body:

```
# backend/src/routes/recommendations.py
@router.post("/recommendations")
def get_recommendations(req: RecommendationRequest):
    input_ratings = [(r.movieId, r.rating) for r in req.ratings]
    recommendations = recommend(input_ratings) # only SELECTs inside
    return {"status": "success", "recommendations": recommendations}
```

6 Cross-cutting: the recommendation algorithm (domain logic)

`recommender.py` is the assignment's **domain** code, not a course networking topic, but it ties several threads together. It is **user-based collaborative filtering**:

1. find DB users who co-rated your movies (one parameterized `SELECT ... IN`),
2. score each with **Pearson correlation** over co-rated items,
3. keep the top-K positive neighbours,
4. predict each unseen movie with the similarity-weighted deviation formula $\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_v \text{sim}(u, v) (r_{v,i} - \bar{r}_v)}{\sum_v |\text{sim}(u, v)|}$,
5. return the top-N, clamped to the `[0.5, 5.0]` MovieLens scale.

Its **course-relevant** properties: it runs only read-only `SELECT`s (stateless), binds every id through `?` placeholders (injection-safe), and opens the DB through the same with `get_db()` context manager (resource cleanup).

7 Concept → code quick-reference table

Lecture	Concept	File · location
L1	Client-server split	frontend/ ↔ backend/
L1	3-tier architecture	frontend / routes+recommender / sqlite
L1	JSON interchange	JSON.stringify / response.json() in index.js
L1/L4	REST stateless	routes/recommendations.py
L2	Statelessness (client state)	let myRatings = {} in index.js
L2	HTTP verbs	@router.get/.post in routes/*.py
L2	Query string + percent-encoding	searchMovies() encodeURIComponent
L2	Path parameter	GET /ratings/{movie_id} in movies.py
L2	MIME / Content-Type	headers: {"Content-Type": "application/json"}
L2	Same-Origin / CORS	CORSMiddleware in main.py
L3	async/await + Promise	callApi() in index.js
L3	fetch API	await fetch(...) in callApi()
L3	Closures / shared state	rateMovie over myRatings
L3	const vs let	top of index.js
L3	Objects as maps	myRatings[id], Object.keys
L3	Strict equality / falsy	if (title === "")
L3	Type conversion	parseFloat, parseInt
L3	Events / DOM	onclick= in index.html, getElementById/innerHTML
L1	HTML markup / CSS presentation	index.html / index.css
L4	FastAPI app + endpoints	main.py, routes/*.py
L4	Pydantic body validation	models.py
L4	Uvicorn / ASGI	uvicorn.run(...) in main.py
L4	SQLite + with + cursor	db.py
L4	DDL / storage classes / PK	setup_db.py CREATE TABLE
L4	Parameterized queries	routes/*.py, recommender.py
L4	executemany / transaction / commit	setup_db.py
L4	JOIN + GROUP BY + fetchall	routes/tags.py

8 Appendix: syllabus concepts deliberately *not* used

Knowing what you **didn't** use (and why) is common exam fodder.

Concept (lecture)	Why not in this project
Cookies / Sessions (L2/L4)	No login/auth; session state kept client-side in myRatings.
HTTP/2, chunked transfer, caching/conditional headers (L2)	App is small; defaults suffice. Not exercised explicitly.
SOAP / WSDL / UDDI (L4)	This is a REST/JSON service, the SOAP alternative.
Java Servlets / JSP / JDBC (L4)	Server side is Python/FastAPI , not the Java stack.
XML / XML Schema / YAML (L1/L4)	Data interchange is JSON only.
PUT / DELETE verbs (L2)	Spec needs only create/read; no update/delete endpoints.
JS classes / inheritance / generators / this rules (L3)	Frontend uses plain functions + closures; no OO classes needed.
addEventListener (L3)	Events wired with inline onclick HTML attributes instead.
Nginx reverse proxy (L4)	Uvicorn is run directly on port 3000 for development.
SSL/TLS (L1)	Local http:// development; no certificates.

These are real, examinable topics — they simply weren't required by the MovieLens assignment. The code shows the **chosen** path (REST + JSON + Python + vanilla JS); this table is the map of the roads not taken.